# Citadel

Local AI chat for Windows · version 1.2.0 · github.com/Gabriel-Castiglia/citadel-windows

Gabriel Castiglia · November 10, 2026 · version 1.2.0 · GPL-2.0 license

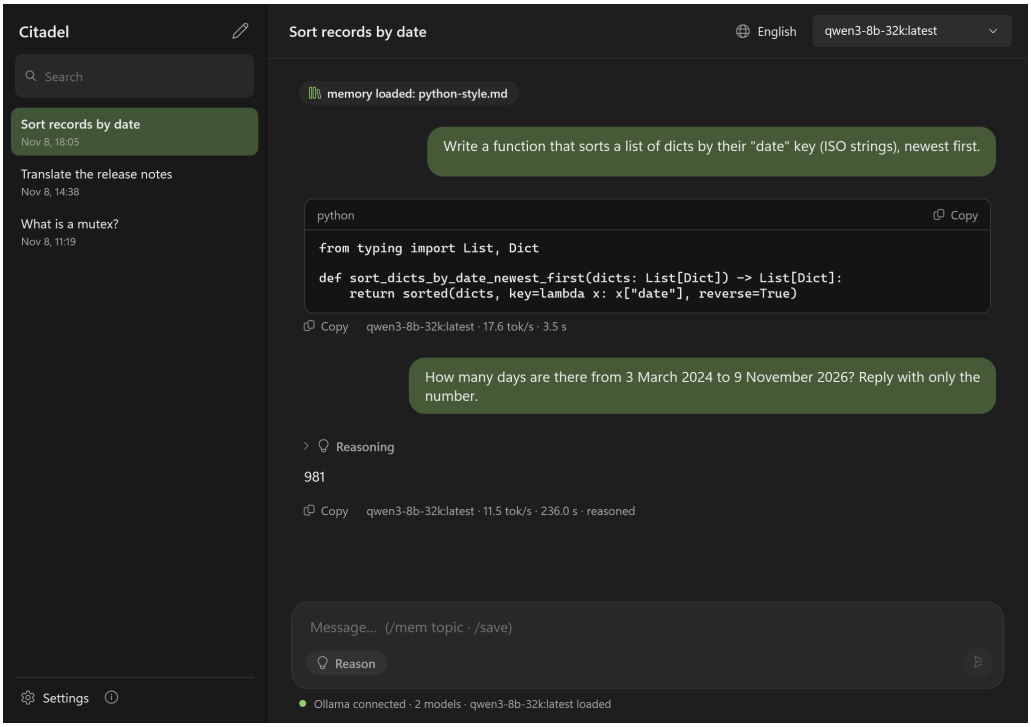
English · Español

Repository: https://github.com/Gabriel-Castiglia/citadel-windows

PDF manual: `docs/Citadel-manual-en.pdf`

Citadel is a chat app for Windows 10 and 11 that talks to language models running on the same computer through Ollama. Nothing you write leaves the computer: the app only connects to Ollama at `127.0.0.1:11434`, unless you choose to reach Ollama on another computer of yours through SSH (see Ollama on another computer). It is written in C# with .NET 10 and WPF, uses the Windows 11 style, dark, light or as Windows, in one of twelve colours, and its interface is in English and Spanish. A macOS version with the same features lives in its own repository, https://github.com/Gabriel-Castiglia/citadel.

Citadel is a chat window, not an agent. The model cannot open files, run commands or browse the web. What it gets is what you type, the text files you attach, a set of rules sent with every request and, when you ask for it, one of your saved memories.




## Requirements

- Windows 11, or Windows 10 version 1803 or later, 64-bit.
- Ollama for Windows, from https://ollama.com/download. Its installer leaves it running in the background and starting with Windows. Then download a model, in PowerShell:

```
ollama pull qwen3:8b
```

Citadel starts with the model `qwen3-8b-32k:latest` selected when it is installed, and otherwise with the first model Ollama lists. That name is Qwen3 8B with a 32,768-token context, created like this:

```
Set-Content Modelfile "FROM qwen3:8b`nPARAMETER num_ctx 32768"
ollama create qwen3-8b-32k -f Modelfile
```

Qwen3 8B needs about 7 GB of memory while loaded. How fast it answers depends on the graphics card: without a card that Ollama supports, the model runs on the processor and answers much more slowly.

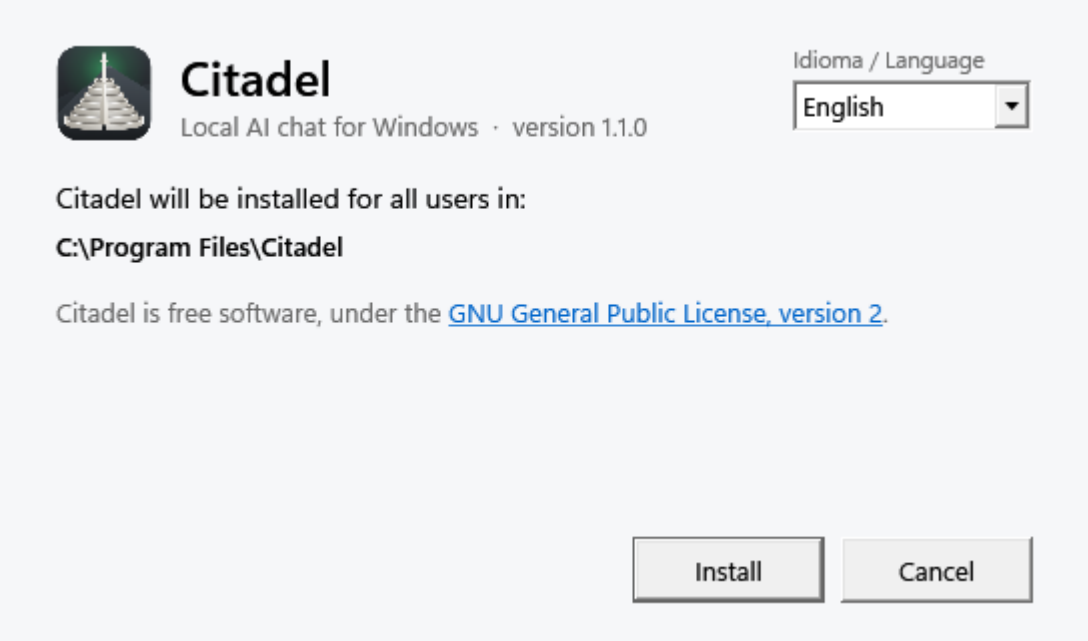
Nothing else has to be installed. Citadel carries its own .NET runtime, and the setup window runs on the .NET Framework that Windows already includes.

## Installation

There are three ways to get Citadel, all built by `scripts\build.ps1` into the `dist` folder:

| File | What it is |
|---|---|
| `citadel-setup.exe` | Setup program, in English or Spanish. The usual way to install. |
| `citadel.msi` | The same installation as a Windows Installer package, for silent installs: `msiexec /i citadel.msi /qn`. |
| `Citadel.exe` | The program on its own; it runs without installation from any folder. |

The setup program opens a window with a language menu in its top right corner, and every page changes language at once. It installs Citadel for all users in `C:\Program Files\Citadel`, which takes administrator rights (Windows asks for them when installation starts), and adds Citadel to the Start menu. The language chosen in the setup window is the one Citadel uses when it starts for the first time.




Running a newer `citadel-setup.exe` updates an installed Citadel and keeps each user's conversations, memories and settings. If Citadel is running, setup closes it before replacing its files. An older version cannot be installed over a newer one.

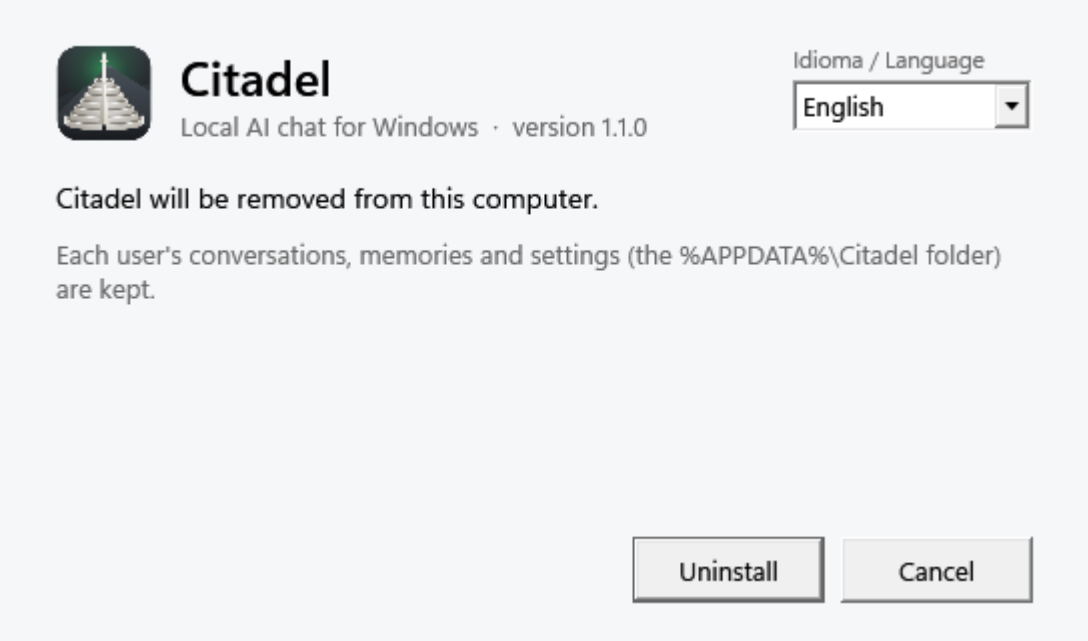
The files are not signed with a commercial certificate, so the first time Windows SmartScreen may show "Windows protected your PC". "More info" → "Run anyway" opens them.

## Uninstallation

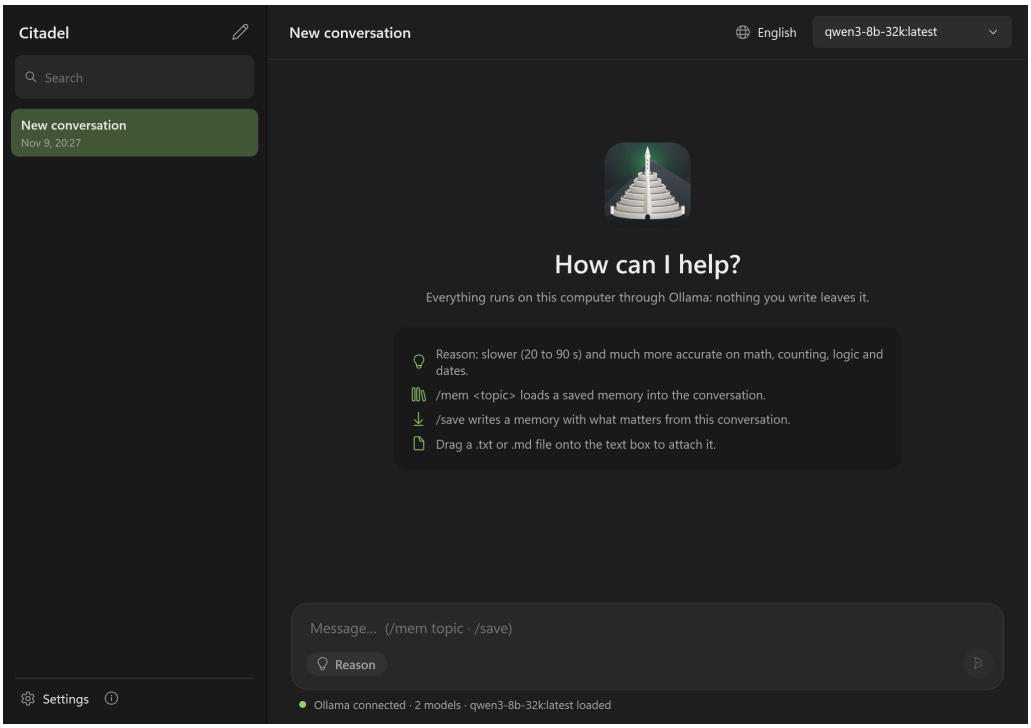
Open Windows Settings → Apps → Installed apps, find Citadel and choose Uninstall. The same setup window opens, with its language menu. It closes Citadel if it is running and removes the program and the Start menu shortcut.

Each user's data, in `%APPDATA%\Citadel`, is kept, so a later installation finds it. To remove it as well, delete that folder.

For an uninstall without windows: `citadel-setup.exe /uninstall /quiet`.




## Using Citadel




Type in the box at the bottom and press Enter to send. Shift+Enter (or Alt+Enter) starts a new line. The answer appears as it is written, rendered as Markdown: headings, lists, tables, quotes, bold, inline code, and code blocks in a monospaced box with their own Copy button. The whole answer can be selected with the mouse. Under each answer are a Copy button for the whole answer, the model, its speed in tokens per second and the seconds it took.

### Conversations

The sidebar lists the conversations, newest first. Ctrl+N or the pencil button starts a new one, and its title is the first line of the first message. Right-click a conversation to rename it or delete it; deleting asks first and removes its file from disk. The search field filters by title, ignoring case and accents.

### Models

The model list at the top right shows every model installed in Ollama and reads them again each time it opens; pick one and the next message goes to it. The choice is remembered, and each answer shows under it the model that wrote it, so one conversation can compare several models. To add a model, install it from a terminal and open the list again:

```
ollama pull gemma3:12b
```

Any chat model in the Ollama library works. Models that cannot reason (Ollama does not list "thinking" among their capabilities) appear in the list with "(no Reason)", and with one of them selected the Reason button is dimmed and its tooltip says why. Citadel asks Ollama for each model's capabilities when it lists them, so a new model is classified as soon as it appears.

Ollama gives each model a context window of a few thousand tokens unless the model says otherwise. For long conversations or large attachments, create a variant with a bigger window, as in Requirements, and select it in the list. A larger window needs more memory.

### Status

The line under the text box says whether Ollama answers and where (on this computer, or on another one through SSH), whether the model is loaded or will load with the next message, what the model is doing (loading, reasoning, writing) with the seconds elapsed, and the speed of the last answer. When Ollama does not answer, the same line says how to start it, or gives the SSH error.

### Reason

The Reason button makes the model think before it answers; it fills with the accent color while it is on. The reasoning goes in a dimmed block above the answer. The block stays folded, showing the latest line while the model thinks, and opens with a click. Reason is slower and much more accurate on arithmetic, counting letters or words, logic and dates. Without it, the model answers at once and is fine for explanations, writing, translation and code. Reason is only available with models that can reason (see Models).

With Reason on, an answer may write up to 8,000 tokens; with it off, up to 3,000. See Technical notes for the measurements behind these choices.

### Stop

While an answer is being written, the send button becomes a Stop button (also Esc). It closes the connection and Ollama stops generating; what was written so far stays, marked "Stopped".

### Attachments

Drag a `.txt` or `.md` file onto the text box to attach its text to the next message; a chip shows the file name and its length, with a button to remove it.

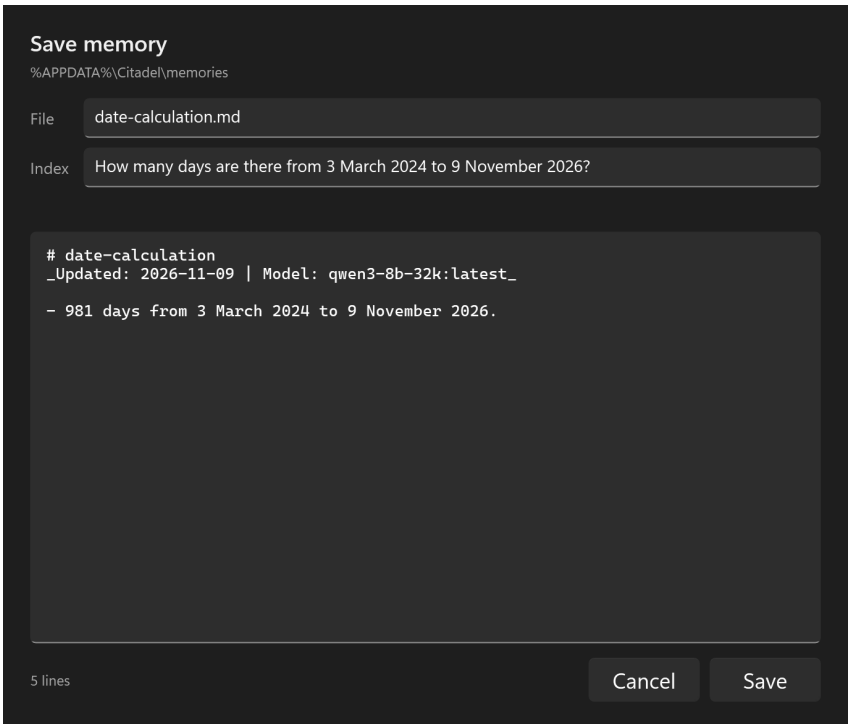
### Memories

A memory is a short Markdown file in the memory folder, listed in that folder's `_index.md` with one line per memory:

```
`kyoto-trip.md` — Kyoto trip in March: dates and bookings
```

The index goes with every request, so the model knows which memories exist. Their content does not, until you load one:

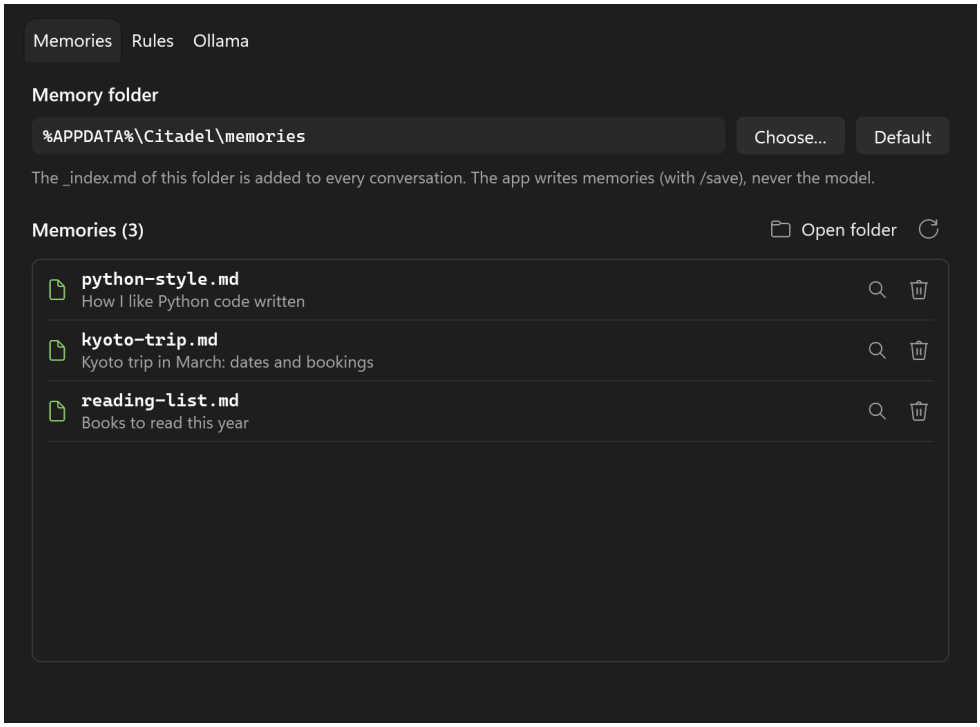
- `/mem <topic>` finds the memory whose index line or file name shares the most words with the topic (accents and case are ignored), and adds it to the conversation. A chip shows "memory loaded:" and the file name. If nothing matches, Citadel says so.
- `/save` asks the model to write a memory from the current conversation: a title, the line `_Updated: YYYY-MM-DD | Model: <model>_`, and the facts worth keeping, in the language of the conversation. A preview opens where the file name, the index line and the text can be edited. If the file already exists, the preview says it will be replaced. Save writes the file and its index line, then reads both back from disk, and only then confirms in the conversation.




Citadel writes the memories, not the model. Small local models often reply "saved" without saving anything, so the model only drafts the text and the app does the writing and the checking.

## Settings

The Settings button at the bottom of the sidebar (or Ctrl+,) opens a window with three tabs.




| Tab | What it holds |
|---|---|
| Memories | The memory folder (Choose…, or Default to go back to `%APPDATA%\Citadel\memories`), the list of memories with their index lines, Open folder, Show in File Explorer for each memory, and Delete, which asks first and removes the file and its index line. |
| Rules | The rules sent at the start of every request, editable. Save (Ctrl+S) writes them; Revert discards the changes; Restore defaults puts back the original rules. |
| Ollama | Where Ollama runs: on this computer, or on another one through SSH (see Ollama on another computer). Apply and test connects at once and lists the models found. |

Any folder can be the memory folder, including one that another tool also uses, as long as it keeps the same `_index.md` format.

## Rules

Every request starts with a system message made of the rules file, the memory index and any memories loaded with `/mem`. The rules file is `%APPDATA%\Citadel\rules.md`, created on the first start with these rules:

1. Answer in the user's language and exactly what was asked; when only the result is asked for, give only the result.
2. Do not invent: if the answer is unknown, after the training data, private, or about something that may not exist, say so in one sentence.
3. Check the premise and say first if it is false.
4. Exact formats: only JSON, only a list or exactly N items means exactly that.
5. Code: one fenced Markdown code block with the complete function, without `input()` or example calls.

6. Translation: keep text between backticks, paths, commands, numbers and names unchanged, and reply only with the translation.
7. The model runs locally through Ollama and nothing leaves the computer.

The last rule exists because a local model asked where it runs tends to answer that it runs in the cloud, which it cannot know.

## Language

The interface is in English and Spanish. The globe button at the top right, next to the model list, shows the current language and opens the language menu; the change applies at once and is remembered. On its first start Citadel uses the language chosen during setup, or English when it runs without installation. The language of the interface does not change the language of the answers: the model answers in the language you write in.

## Theme and colour

Settings → Appearance chooses the look of the window: as Windows (light or dark, like your other apps), light or dark, which is the default. Below it, twelve colours change the buttons, the selection and the glow behind the citadel in the icon of the window and the taskbar while Citadel is open; green is the original. The Start menu and desktop shortcuts keep the green icon. The Mac version has the same choices.

## Ollama on another computer

A model too big for this computer can run on another computer of yours, and Citadel can use it through SSH. Citadel starts the `ssh` command that comes with Windows, with a tunnel from a free local port to port 11434 of that computer, sends every request through it, and closes it on exit. Ollama's port is never opened to the network: the other computer only has to accept SSH. Conversations, rules and memories stay on this computer; the other one receives each request as it is sent, as Ollama always does.

On the computer that runs the model:

1. Install Ollama and the models, as in Requirements. Ollama only needs to listen on `127.0.0.1`, its default.

2. Turn on the SSH server: on a Mac, System Settings → General → Sharing → Remote Login; on Windows, Settings → System → Optional features → OpenSSH Server, then start the `sshd` service.

3. Authorize this computer's public key (`%USERPROFILE%\.ssh\id_ed25519.pub`; create it with `ssh-keygen -t ed25519` if it does not exist). In the user's `~/.ssh/authorized_keys` on the other computer, a line like this lets that key forward Ollama's port and nothing else (no shell, no commands, no other ports):

   ```
   restrict,port-forwarding,permitopen="127.0.0.1:11434",command="/usr/bin/false" ssh-ed25519 AAAA… name
   ```

> On a Windows server that path is `C:\Users\<user>\.ssh\authorized_keys` for a standard user, or `C:\ProgramData\ssh\administrators_authorized_keys` for an administrator, and the forced command is `command="cmd /c exit"`.

On this computer, open Settings → Ollama, choose "On another computer, through SSH", type the destination (`user@host`, or a `Host` name from `%USERPROFILE%\.ssh\config`) and click Apply and test. The result line lists the models found there, and the status line under the text box says "Ollama on (SSH)". Citadel never asks for a password (it runs ssh in batch mode), so the key must work without one, or be loaded in the OpenSSH agent. The first connection to a new host records its key in `known_hosts`; a later change of that key makes the connection fail, as in any SSH client. If the connection drops, Citadel opens it again with the next request.

## Keyboard

| Keys | Action |
|---|---|
| Enter | Send |
| Shift+Enter, Alt+Enter | New line |
| Ctrl+N | New conversation |
| Esc | Stop the answer |
| Ctrl+, | Settings |
| Ctrl+S | Save the rules (Rules tab) |

## Files

| What | Where |
|---|---|
| Installed program | `C:\Program Files\Citadel`, with `LICENSE.txt`, `README.txt` and `README.es.txt` |
| Conversations | `%APPDATA%\Citadel\conversations\`, one JSON file each |
| Rules | `%APPDATA%\Citadel\rules.md` |
| Memories (default folder) | `%APPDATA%\Citadel\memories\`, with `_index.md` |
| Settings (language, model, Reason, memory folder, SSH destination, window position) | `%APPDATA%\Citadel\settings.json` |
| Error log, if anything fails | `%APPDATA%\Citadel\citadel.log`, kept under 512 KB |

Files are written in UTF-8 without a byte order mark. Conversations are plain JSON in the same format as the macOS version, so they can be copied between the two.

## Building from source

With the .NET 10 SDK installed:

```
dotnet build src\Citadel\Citadel.csproj -c Release
powershell -ExecutionPolicy Bypass -File scripts\build.ps1
```

The first command builds the program. The second builds everything in `dist`: the stand-alone `Citadel.exe`, `citadel.msi` and `citadel-setup.exe`; the WiX Toolset 5 is restored automatically as a NuGet package. The version is set in one place, `src\Citadel\Citadel.csproj`, and the installers and the About window read it from there.

The program has two modes for maintenance. Both take `--data-folder <folder>` to use a folder other than `%APPDATA%\Citadel`, so they never touch real conversations.

- `Citadel.exe --self-test --memory-folder <empty folder>` runs the main features end to end against Ollama and prints one PASS or FAIL line per check: listing the models, an answer with Reason off and with Reason on, Stop, `/save` (file, index line, no byte order mark) and `/mem`. It refuses to run against a memory folder in use. Its output can be read with `Start-Process Citadel.exe -ArgumentList ... -Wait -NoNewWindow -RedirectStandardOutput out.txt`.
- `Citadel.exe --screenshot <file.png> --window main|settings|about|save --language en|es` draws one window to an image and exits.

`scripts\screenshots.ps1 -Demo <folder>` renders the images in `docs\images` with demonstration conversations and memories, `scripts/manual.sh` (on macOS, with pandoc and Google Chrome) builds the PDF manuals and `README.txt` and `README.es.txt` from this file and `README.es.md`, and `scripts/make-icon.sh` (on macOS) builds `src\Citadel\citadel.ico` from `icon\citadel-1024.png`.

| Folder | Contents |
|---|---|
| `src/Citadel` | The program. |
| `installer/Msi` | The MSI package (WiX). |
| `installer/SetupUI` | The setup window, in English and Spanish (.NET Framework 4.7.2, WPF). |
| `installer/Setup` | The setup program that carries the setup window and the MSI (WiX bundle). |
| `scripts` | Build, images, icon and manuals. |
| `icon` | The icon, as SVG and as a 1024-pixel PNG. |
| `docs` | PDF manuals, images and the manual stylesheet. |

## Technical notes

- The windows use WPF's Fluent theme (`Application.ThemeMode`), dark or light as chosen, which .NET 10 still marks as experimental (warning WPF0001, silenced in the project file). The accent color is the one chosen in Settings → Appearance (green by default).

- Answers are shown in a read-only rich text box holding a flow document, so a whole answer can be selected and copied. Code blocks are embedded boxes with their own Copy button.
- Answers arrive from `/api/chat` as one JSON object per line. Stop cancels the request, which closes the connection; Ollama then cancels the generation, as its log shows (`srv stop: cancel task`). Models that write their reasoning between `<think>` tags instead of in the `thinking` field are handled too.
- On 15 hard questions of counting, arithmetic, logic and dates, asked twice each, Qwen3 8B answered 9 of 30 correctly without reasoning and 26 of 30 with it. With reasoning, the median answer took 38 seconds and the slowest 172, on a computer where the model writes about 17 tokens per second. Sometimes the model loops in its reasoning until it reaches the 8,000-token cap; Stop ends it.
- `/save` asks the model for JSON (`"format": "json"`) without reasoning and with a 3,000-token cap. Citadel then makes the file name kebab-case, rewrites the `_Updated` line with the real date and model, and replaces the old index line of the same file instead of adding a second one.
- Ollama answers a request with `"think": true` to a model without that capability with an error ("does not support thinking"). Citadel reads the capabilities from `/api/show` and sends `"think": false` to those models, whatever the state of the Reason button.
- The SSH tunnel runs `ssh -N` with `BatchMode=yes`, `ExitOnForwardFailure=yes` and keepalives every 15 seconds; Citadel waits until the local end accepts connections before sending. ssh.exe runs in a job object that Windows closes with Citadel, so the tunnel ends even if Citadel is killed.

## Credits

Citadel uses Ollama (MIT license) to run the models, which are downloaded separately under their own licenses. Qwen3 is published by the Qwen team under the Apache 2.0 license. The installers are built with the WiX Toolset (MS-RL license).

## Version history

See `CHANGELOG.md`.

## License

Copyright (C) 2026 Gabriel Castiglia.

Citadel is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License, version 2 only (GPL-2.0-only), as published by the Free Software Foundation. It is distributed WITHOUT ANY WARRANTY. See `LICENSE` for the full text.